



Matrizes

O que é uma Matriz?

- Organização de dados em linhas e colunas

- Exemplo visual:

```
matriz = [[1, 2], [3, 4], [5, 6]]
```

- Matrizes são listas dentro de listas
- Cada elemento da lista principal é uma linha da matriz.



```
matriz = [[1, 2], [3, 4], [5, 6]]
```

Acessando Elementos

Sintaxe: `matriz[linha][coluna]`

```
print(matriz[0][1]) # Resultado: 2
```

Modificando Elementos

```
matriz[1][0] = 99  
print(matriz)
```

Atualiza o valor na linha 1, coluna 0.





Percorrendo Matrizes com for

Percorrer linhas e colunas:

```
for linha in matriz:  
    for elemento in linha:  
        print(elemento, end=" ")  
    print()
```



Exemplo Prático

- Criar uma matriz 2x2 preenchida manualmente

```
matriz = [[0, 0], [0, 0]]  
matriz[0][0] = int(input("Digite um número: "))  
matriz[0][1] = int(input("Digite um número: "))  
matriz[1][0] = int(input("Digite um número: "))  
matriz[1][1] = int(input("Digite um número: "))  
print(matriz)
```



Manipulação de listas (Vetores)

Manipulação Avançada de Listas

- Vamos aprender como manipular listas de maneira mais eficiente.
- Exploraremos métodos e técnicas úteis para trabalhar com listas em Python.



Métodos Básicos de Listas

Lembrando: já vimos como criar, acessar e modificar listas. Agora, vamos explorar novos métodos.

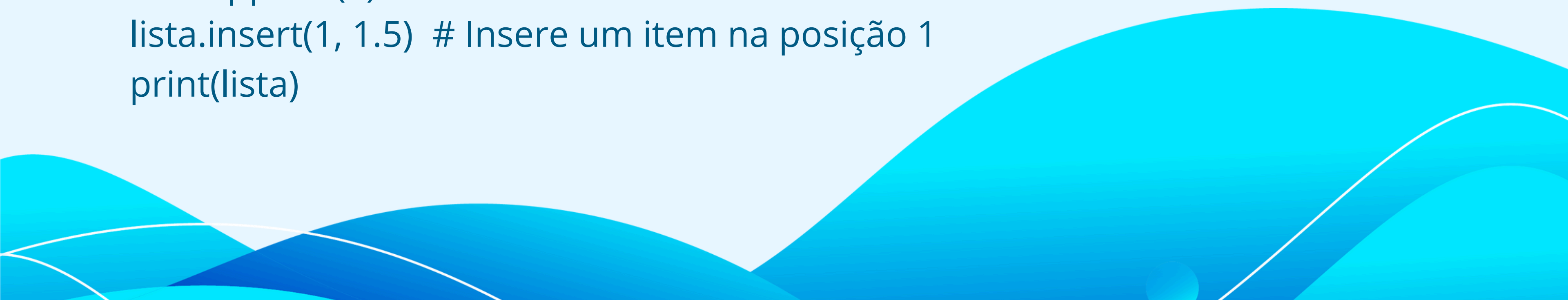
insert()

```
lista = [1, 2, 3]
```

```
lista.append(4) # Adiciona um item ao final
```

```
lista.insert(1, 1.5) # Insere um item na posição 1
```

```
print(lista)
```





Método pop()

pop(): Remove e retorna o item da lista em uma posição específica (ou do final, por padrão).

```
lista = [1, 2, 3, 4]
```

```
item = lista.pop(1) # Remove o item na posição 1 (valor 2)
```

```
print(lista) # Saída: [1, 3, 4]
```

```
print(item) # Saída: 2
```





Método reverse()

reverse(): Inverte a ordem dos elementos da lista.

```
lista = [1, 2, 3, 4]
```

```
lista.reverse()
```

```
print(lista) # Saída: [4, 3, 2, 1]
```





Fatiamento (Slicing)

Fatiamento: A técnica de acessar um subconjunto da lista usando a sintaxe [início:fim].

```
lista = [1, 2, 3, 4, 5]  
sublista = lista[1:4] # Pega do índice 1 até o 3 (não inclui o 4)  
print(sublista) # Saída: [2, 3, 4]
```

```
lista[:3] # Primeiros 3 elementos  
lista[3:] # Do índice 3 até o final  
lista[::-1] # Inverte a lista
```





Método sort()

sort(): Ordena os itens da lista em ordem crescente (ou decrescente, com o parâmetro reverse=True).

```
lista = [4, 2, 9, 1]
```

```
lista.sort() # Ordenação crescente
```

```
print(lista) # Saída: [1, 2, 4, 9]
```

```
lista.sort(reverse=True) # Ordenação decrescente
```

```
print(lista) # Saída: [9, 4, 2, 1]
```





Método extend()

extend(): Adiciona todos os itens de uma lista a outra.

```
lista1 = [1, 2]
```

```
lista2 = [3, 4]
```

```
lista1.extend(lista2)
```

```
print(lista1) # Saída: [1, 2, 3, 4]
```





Método index()

index(): Retorna o índice da primeira ocorrência de um valor na lista.

```
lista = [10, 20, 30, 40]  
indice = lista.index(30)  
print(indice) # Saída: 2
```





Método count()

count(): Retorna o número de vezes que um valor aparece na lista.

```
lista = [1, 2, 2, 3, 2]  
quantidade = lista.count(2)  
print(quantidade) # Saída: 3
```

